

Cross-Domain State Preservation Functor

Jinwook Kim (Jay)

April 7, 2026

Abstract

We present a generic, reusable framework for verifying that state transitions are faithfully preserved when synchronized across heterogeneous domains (blockchains, off-chain ledgers, etc.), and that the synchronization process remains live in the presence of Byzantine faults.

The framework combines two complementary properties. For *safety*, the core abstraction is a hierarchy of Isabelle/HOL locales: `state_machine` defines finite-state deterministic transition systems with terminal states; `state_preservation` captures the naturality (homomorphism) condition ensuring that a synchronization map commutes with transitions; `symmetric_state_preservation` extends this to bidirectional synchronization with inverse roundtrip guarantees; and `multi_domain_preservation` generalizes to N domains sharing the same abstract state machine, with a cross-domain consistency theorem. For *liveness* under Byzantine faults, three additional locales are introduced: `priority_system` guarantees deterministic resolution of conflicting requests via a total order on priority keys; `deadlock_free_locking` guarantees that timeout-based forced release prevents indefinite locking; and `fair_leader_system` guarantees starvation freedom under fair leader scheduling, formulated as an assume-guarantee abstraction of probabilistic VRF-based leader election.

As a practical application, we instantiate this framework with the regulatory state model of Oraclizer, a state synchronization oracle for tokenized assets across blockchain networks and off-chain ledgers. The model comprises five states (ACTIVE, FROZEN, SEIZED, CONFISCATED, RESTRICTED) and seven actions. The safety instantiation proves that a synchronization protocol with preemptive locking preserves cross-chain regulatory consistency (the *regulatory homomorphism* theorem), that the global validity invariant is maintained across synchronization events (*valid state preservation*), and that the generic multi-domain locale applies parametrically to the regulatory model. The liveness instantiation models a Byzantine consensus system with $f < n/3$ malicious nodes, instantiates the three liveness locales with regulatory authority levels and a four-component priority key, and combines safety and liveness in a single *combined safety liveness* theorem that discharges the honest-node assumption of the safety proof under Byzantine faults.

The design is inspired by Lochbihler’s Merkle Functor (AFP entry `ADS_Functor`), which abstracted authenticated data structures into composable building blocks. This entry extends that pattern to a different axis of abstraction: cross-domain state preservation targeting synchronization correctness rather than data structure integrity, together with the liveness guarantees needed for that synchronization to actually make progress under Byzantine faults.

Contents

1	Overview	2
1.1	Theory Dependency Structure	3
1.2	Design Decisions	3
2	Generic State Machine Locale	4
3	Cross-Domain State Preservation Locale	5
4	Symmetric State Preservation	7
5	Multi-Domain State Preservation	8
6	Regulatory State and Action Definitions	10
7	Fundamental Properties of the Transition Function	11
7.1	Invariant I1: CONFISCATED is terminal	11
7.2	Invariant I2: CONFISCATE reaches CONFISCATED from any non-terminal state	11
7.3	Invariant I3: Determinism (inherent from function definition)	12
7.4	SEIZED to FROZEN exclusion	12
7.5	FROZEN to RESTRICTED exclusion	12
7.6	Transition completeness: every non-terminal state has at least one valid action	12
7.7	Reachability from ACTIVE	13
7.8	Return paths to ACTIVE	13
8	State Machine Instance	13
9	Asset and Global State Modeling	14
10	State Accessors and Predicates	15
11	Global State Validity	15

12 Synchronization Protocol	16
12.1 Lock acquisition	16
12.2 State update across chains	16
12.3 The synchronization function	17
13 Synchronization Properties	17
13.1 Idempotency	17
13.2 Sync isolation: other assets are not affected	18
13.3 Sync correctness: source chain updated	18
13.4 Properties of update_all_chains	18
13.5 Lock mutual exclusion	19
14 Main Theorem: Cross-Chain Regulatory Consistency	20
15 Valid State Preservation	22
15.1 Part 1: consistent_state preservation	22
15.2 Part 2: no_locked_without_reason preservation	25
15.3 The valid_state preservation theorem	26
16 Multi-Domain Locale Instantiation	27
17 Priority-Based Deterministic Selection	29
18 Deadlock-Free Locking with Timeout	31
19 Fair Leader Starvation Freedom	32
20 Authority Level and Action Severity	36
21 Priority Key	37
22 Extended Message Type	37
23 Priority Construction	37
24 Node and System Model	38
25 D-quencer System Locale	38
26 Priority System Instantiation	39
27 BFT Consensus Abstraction	40
28 Deadlock-Free Locking Instantiation	41
29 Fair Leader Starvation Freedom Instantiation	42

30 Combined Safety and Liveness	43
31 Acknowledgments	44

1 Overview

This AFP entry consists of four theory files organized as two “generic locale + regulatory instance” pairs, one for safety and one for liveness:

State_Preservation.thy defines domain-independent locales for state machines, pairwise state preservation (with naturality), symmetric bidirectional preservation, and multi-domain preservation. All locales are parameterized over abstract types and can be instantiated with any domain that has finite states, deterministic transitions, and optional terminal states.

Regulatory_Instance.thy instantiates the generic safety framework with a concrete regulatory state model. It defines the five regulatory states and seven regulatory actions, proves fundamental properties (terminal absorption, universal reachability of confiscation, transition exclusions with legal justification), models a synchronization protocol with preemptive locking, and proves the main safety theorems: *regulatory homomorphism* (cross-chain consistency after sync), *valid state preservation* (sync maintains the global validity invariant), and *multi-domain parametric instantiation*.

Priority_Resolution.thy defines domain-independent locales for liveness properties under Byzantine faults: **priority_system** for deterministic selection from a finite message set with injective priorities, **deadlock_free_locking** for timeout-based forced release of locks, and **fair_leader_system** for starvation freedom under periodic honest leader scheduling. The locales are parameterized over abstract types and impose no dependency on the regulatory domain; they are reusable for any distributed system requiring deterministic conflict resolution, deadlock freedom, or fair scheduling.

DQuencer_Instance.thy instantiates the generic liveness framework with the regulatory domain. It defines authority levels (REGIONAL, NATIONAL, INTERNATIONAL), a four-component priority key with proven injectivity, a Byzantine fault model with the BFT threshold $n \geq 3f + 1$, and instantiates each of the three liveness locales for the regulatory consensus system. It proves the main liveness theorems (*select highest deterministic*, *deadlock freedom*, *eventual completion*) and concludes with the *combined safety liveness* theorem that links the liveness results back to the synchronization function defined in

`Regulatory_Instance.thy`, discharging the honest-node assumption of the safety proof under Byzantine faults.

1.1 Theory Dependency Structure

The four theories form two independent generic-instance pairs that meet at `DQuencer_Instance.thy`: `State_Preservation.thy` is imported by `Regulatory_Instance.thy`; `Priority_Resolution.thy` depends only on `Main` and is deliberately kept independent of the regulatory domain to preserve reusability; `DQuencer_Instance.thy` imports both `Priority_Resolution.thy` and `Regulatory_Instance.thy`, serving as the join point where the liveness framework is applied to the regulatory model and combined with the safety results.

1.2 Design Decisions

The regulatory state model incorporates several deliberate design decisions based on legal precedence and operational considerations:

- `RECOVER` and `LIQUIDATE` are excluded from the state machine as they involve force transfers or external DEX interactions rather than state transitions.
- The direct transition `SEIZED` $\rightarrow$ `FROZEN` is excluded because seizure is a strictly stronger legal constraint than freezing; relaxation requires explicit release first.
- The direct transition `FROZEN` $\rightarrow$ `RESTRICTED` is excluded to enforce passage through `ACTIVE`.
- Preemptive locking prevents concurrent regulatory actions on the same asset, *not* double-spending (which is handled at a different layer).

The liveness model adopts an assume-guarantee approach for starvation freedom: rather than directly modeling the probabilistic guarantees of VRF-based leader election (which would require extensive probability theory infrastructure), the `fair_leader_system` locale assumes that within any `fairness_bound` consecutive epochs, at least one epoch has an honest leader. Under $f < n/3$ Byzantine faults, the probability that a Byzantine leader is elected k consecutive times is $(f/n)^k$, which decreases exponentially. The fairness bound captures this as a deterministic upper bound, following standard assume-guarantee reasoning practice.

```
theory State-Preservation
  imports Main
begin
```

2 Generic State Machine Locale

A state machine with finite state set, deterministic partial transition function, and optional terminal states. Terminal states accept no transitions.

```

locale state-machine =
  fixes states :: 's set
    and actions :: 'a set
    and transition :: 's  $\Rightarrow$  'a  $\Rightarrow$  's option
    and terminal :: 's set
  assumes finite-states: finite states
    and finite-actions: finite actions
    and terminal-subset: terminal  $\subseteq$  states
    and terminal-absorbing:  $\llbracket s \in \text{terminal}; a \in \text{actions} \rrbracket \implies \text{transition } s \ a = \text{None}$ 
  and transition-closed:  $\llbracket s \in \text{states}; a \in \text{actions}; \text{transition } s \ a = \text{Some } s' \rrbracket \implies s' \in \text{states}$ 
  and transition-domain:  $\llbracket s \notin \text{states} \rrbracket \implies \text{transition } s \ a = \text{None}$ 
begin

```

Determinism is inherent: transition is a function, not a relation.

lemma transition-deterministic:

$\text{transition } s \ a = \text{Some } s1 \implies \text{transition } s \ a = \text{Some } s2 \implies s1 = s2$

by simp

Sequential composition of actions (partial, returns None on first failure).

fun apply-actions :: 's $\Rightarrow$ 'a list $\Rightarrow$'s option **where**

$\text{apply-actions } s \ [] = \text{Some } s$

| $\text{apply-actions } s \ (a \ \# \ as) = (\text{case } \text{transition } s \ a \ \text{of}$

$\text{None} \Rightarrow \text{None}$

| $\text{Some } s' \Rightarrow \text{apply-actions } s' \ as)$

lemma apply-actions-closed:

$\llbracket s \in \text{states}; \forall a \in \text{set } as. a \in \text{actions}; \text{apply-actions } s \ as = \text{Some } s' \rrbracket \implies s' \in \text{states}$

by (induction as arbitrary: s) (auto split: option.splits dest: transition-closed)

Note: $\text{apply-actions } ?s \ [] = \text{Some } ?s$ already provides the simp rule $\text{apply-actions } s \ [] = \text{Some } s$, so no separate [simp] lemma is needed.

lemma apply-actions-terminal:

$\llbracket s \in \text{terminal}; a \in \text{actions} \rrbracket \implies \text{apply-actions } s \ (a \ \# \ as) = \text{None}$

by (simp add: terminal-absorbing)

end

3 Cross-Domain State Preservation Locale

Two state machines connected by a synchronization map. The key property (naturality / homomorphism) states that synchronizing after a local

transition yields the same result as transitioning after synchronization.

More precisely: for state machines (S_src, T_src) and (S_tgt, T_tgt) connected by sync (mapping source states to target states), the following diagram commutes:

$$\begin{array}{ccc} s_src -T_src(a) \rightarrow s_src' & \begin{array}{c} | \\ | \end{array} & \begin{array}{c} \text{sync} \quad \text{sync} \\ | \quad | \end{array} & \begin{array}{c} v \\ v \end{array} & s_tgt -T_tgt(a) \rightarrow s_tgt' \end{array}$$

That is: $\text{sync } (T_src \ s \ a) = T_tgt \ (\text{sync } s) \ a$

This is the structural preservation guarantee: a transition performed at the source domain, when synchronized, produces the same result as performing the corresponding transition at the target domain.

locale *state-preservation* =

source: state-machine $states_s \ actions_s \ transition_s \ terminal_s$ +

target: state-machine $states_t \ actions_t \ transition_t \ terminal_t$

for $states_s :: 's \text{ set}$ **and** $actions_s :: 'a \text{ set}$

and $transition_s :: 's \Rightarrow 'a \Rightarrow 's \text{ option}$

and $terminal_s :: 's \text{ set}$

and $states_t :: 't \text{ set}$ **and** $actions_t :: 'b \text{ set}$

and $transition_t :: 't \Rightarrow 'b \Rightarrow 't \text{ option}$

and $terminal_t :: 't \text{ set}$ +

fixes $state\text{-}map :: 's \Rightarrow 't$

and $action\text{-}map :: 'a \Rightarrow 'b$

assumes *state-map-well-defined*: $s \in states_s \implies state\text{-}map \ s \in states_t$

and *action-map-well-defined*: $a \in actions_s \implies action\text{-}map \ a \in actions_t$

and *terminal-preservation*: $s \in terminal_s \implies state\text{-}map \ s \in terminal_t$

— The naturality / homomorphism condition:

and *naturality*:

$\llbracket s \in states_s; a \in actions_s; transition_s \ s \ a = Some \ s' \rrbracket$

$\implies transition_t \ (state\text{-}map \ s) \ (action\text{-}map \ a) = Some \ (state\text{-}map \ s')$

and *naturality-none*:

$\llbracket s \in states_s; a \in actions_s; transition_s \ s \ a = None \rrbracket$

$\implies transition_t \ (state\text{-}map \ s) \ (action\text{-}map \ a) = None$

begin

The fundamental theorem: state preservation extends to sequences of actions. If a sequence of transitions is valid at the source, the mapped sequence is valid at the target with the mapped final state.

theorem *sequential-preservation*:

assumes $s \in states_s$

and $\forall a \in as. a \in actions_s$

and $source.apply\text{-}actions \ s \ as = Some \ s'$

shows $target.apply\text{-}actions \ (state\text{-}map \ s) \ (map \ action\text{-}map \ as) = Some \ (state\text{-}map \ s')$

using *assms*

proof (*induction as arbitrary: s*)

case *Nil*

then show *?case* **by** *simp*

next

case (*Cons a as*)

```

then obtain  $s\text{-mid}$  where
   $\text{step: transition}_s s a = \text{Some } s\text{-mid}$  and
   $\text{rest: source.apply-actions } s\text{-mid } as = \text{Some } s'$ 
  by (auto split: option.splits)
from  $\text{Cons.premis}(1) \text{ Cons.premis}(2)$  step have  $s\text{-mid} \in \text{states}_s$ 
  using  $\text{source.transition-closed}$  by auto
from  $\text{Cons.premis}(1) \text{ Cons.premis}(2)$  step
  have  $\text{mapped-step: transition}_t (\text{state-map } s) (\text{action-map } a) = \text{Some } (\text{state-map } s\text{-mid})$ 
  using  $\text{naturality}$  by auto
from  $\langle s\text{-mid} \in \text{states}_s \rangle \text{ Cons.premis}(2)$  rest
have  $\text{target.apply-actions } (\text{state-map } s\text{-mid}) (\text{map action-map } as) = \text{Some } (\text{state-map } s')$ 
  using  $\text{Cons.IH}$  by auto
with  $\text{mapped-step}$  show  $?case$  by simp
qed

theorem sequential-preservation-none:
  assumes  $s \in \text{states}_s$ 
  and  $\forall a \in \text{set } as. a \in \text{actions}_s$ 
  and  $\text{source.apply-actions } s as = \text{None}$ 
  shows  $\text{target.apply-actions } (\text{state-map } s) (\text{map action-map } as) = \text{None}$ 
  using  $\text{assms}$ 
proof (induction as arbitrary: s)
  case Nil
  then show  $?case$  by simp
next
  case ( $\text{Cons } a as$ )
  show  $?case$ 
  proof (cases transitions s a)
    case None
    then have  $\text{transition}_t (\text{state-map } s) (\text{action-map } a) = \text{None}$ 
    using  $\text{naturality-none Cons.premis}(1) \text{ Cons.premis}(2)$  by auto
    then show  $?thesis$  by simp
  next
    case ( $\text{Some } s\text{-mid}$ )
    then have  $\text{mid-in: } s\text{-mid} \in \text{states}_s$ 
    using  $\text{source.transition-closed Cons.premis}(1) \text{ Cons.premis}(2)$  by auto
    from  $\text{Some Cons.premis}(3)$  have  $\text{source.apply-actions } s\text{-mid } as = \text{None}$ 
    by simp
    then have  $\text{ih: target.apply-actions } (\text{state-map } s\text{-mid}) (\text{map action-map } as) = \text{None}$ 
    using  $\text{Cons.IH mid-in Cons.premis}(2)$  by auto
    from  $\text{Some}$  have  $\text{transition}_t (\text{state-map } s) (\text{action-map } a) = \text{Some } (\text{state-map } s\text{-mid})$ 
    using  $\text{naturality Cons.premis}(1) \text{ Cons.premis}(2)$  by auto
    with  $\text{ih}$  show  $?thesis$  by simp
  qed
qed

```

Terminal states are preserved: if a source state is terminal, its image is terminal.

corollary *terminal-image-absorbing*:

assumes $s \in \text{terminal}_s$ **and** $a \in \text{actions}_s$

shows $\text{transition}_t (\text{state-map } s) (\text{action-map } a) = \text{None}$

proof –

from $\text{assms}(1)$ **have** $s \in \text{states}_s$ **using** *source.terminal-subset* **by** *auto*

from assms **have** $\text{transition}_s s a = \text{None}$

using *source.terminal-absorbing* **by** *auto*

then show *?thesis*

using *naturality-none* $\langle s \in \text{states}_s \rangle$ $\text{assms}(2)$ **by** *auto*

qed

end

4 Symmetric State Preservation

For bidirectional synchronization, both directions must preserve structure. This locale composes two **state_preservation** instances with inverse maps, establishing that synchronization in either direction is structure-preserving.

locale *symmetric-state-preservation* =

forward: state-preservation

$\text{states}_s \text{ actions}_s \text{ transition}_s \text{ terminal}_s$

$\text{states}_t \text{ actions}_t \text{ transition}_t \text{ terminal}_t$

$\text{state-map} \text{ action-map} +$

backward: state-preservation

$\text{states}_t \text{ actions}_t \text{ transition}_t \text{ terminal}_t$

$\text{states}_s \text{ actions}_s \text{ transition}_s \text{ terminal}_s$

$\text{state-map-inv} \text{ action-map-inv}$

for $\text{states}_s :: 's \text{ set}$ **and** $\text{actions}_s :: 'a \text{ set}$

and $\text{transition}_s :: 's \Rightarrow 'a \Rightarrow 's \text{ option}$

and $\text{terminal}_s :: 's \text{ set}$

and $\text{states}_t :: 't \text{ set}$ **and** $\text{actions}_t :: 'b \text{ set}$

and $\text{transition}_t :: 't \Rightarrow 'b \Rightarrow 't \text{ option}$

and $\text{terminal}_t :: 't \text{ set}$

and $\text{state-map} :: 's \Rightarrow 't$ **and** $\text{action-map} :: 'a \Rightarrow 'b$

and $\text{state-map-inv} :: 't \Rightarrow 's$ **and** $\text{action-map-inv} :: 'b \Rightarrow 'a +$

assumes *roundtrip-state-src*: $s \in \text{states}_s \implies \text{state-map-inv} (\text{state-map } s) = s$

and *roundtrip-state-tgt*: $t \in \text{states}_t \implies \text{state-map} (\text{state-map-inv } t) = t$

and *roundtrip-action-src*: $a \in \text{actions}_s \implies \text{action-map-inv} (\text{action-map } a) = a$

and *roundtrip-action-tgt*: $b \in \text{actions}_t \implies \text{action-map} (\text{action-map-inv } b) = b$

begin

Under symmetric preservation with roundtrip maps, **state_map** is injective on the state set – distinct source states map to distinct target states. This guarantees no information loss during synchronization.

lemma *state-map-injective*:

```

     $\llbracket s1 \in \text{states}_s; s2 \in \text{states}_s; \text{state-map } s1 = \text{state-map } s2 \rrbracket \implies s1 = s2$ 
    using roundtrip-state-src by metis
end

```

5 Multi-Domain State Preservation

Generalization to N domains sharing the same abstract state machine. This models scenarios where a single asset exists on multiple domains (e.g., blockchain networks), all of which must reflect the same state.

The key property: if one domain transitions, the synchronization brings ALL other domains to the same resulting state.

We model this as: all domains are instances of the same abstract state machine, connected by identity-like state/action maps. The "same abstract state machine" assumption holds when the protocol enforces a uniform state model across all domains.

```

locale multi-domain-preservation =
  fixes domains :: 'd set
  and states :: 's set
  and actions :: 'a set
  and transition :: 's  $\Rightarrow$  'a  $\Rightarrow$  's option
  and terminal :: 's set
  and domain-state :: 'd  $\Rightarrow$  'id  $\Rightarrow$  's option
    — Current state of asset 'id in domain 'd
  assumes fin-domains: finite domains
  and sm: state-machine states actions transition terminal
  and consistent-init:
    — Precondition: all domains holding an asset agree on its state
     $\llbracket d1 \in \text{domains}; d2 \in \text{domains};$ 
      domain-state d1 aid = Some s1; domain-state d2 aid = Some s2  $\rrbracket$ 
     $\implies s1 = s2$ 
begin

```

Which domains hold a given asset.

```

definition connected-domains :: 'id  $\Rightarrow$  'd set where
  connected-domains aid = {d  $\in$  domains. domain-state d aid  $\neq$  None}

```

The global consensus state of an asset (well-defined by `consistent_init`).

```

definition consensus-state :: 'id  $\Rightarrow$  's option where
  consensus-state aid =
    (if connected-domains aid = {} then None
     else domain-state (SOME d. d  $\in$  connected-domains aid) aid)

```

After synchronizing an action on an asset, all connected domains reflect the new state. This is the cross-domain consistency theorem.

```

definition sync-all ::
  'd  $\Rightarrow$  'a  $\Rightarrow$  'id  $\Rightarrow$  ('d  $\Rightarrow$  'id  $\Rightarrow$  's option)  $\Rightarrow$  ('d  $\Rightarrow$  'id  $\Rightarrow$  's option) option

```

where

```

sync-all source action aid ds =
  (case ds source aid of
    None  $\Rightarrow$  None
  | Some s  $\Rightarrow$ 
    (case transition s action of
      None  $\Rightarrow$  None
    | Some s'  $\Rightarrow$ 
      Some ( $\lambda d$  aid'.
        if aid' = aid  $\wedge d \in$  connected-domains aid
        then Some s'
        else ds d aid'))))

```

The cross-domain consistency theorem: after **sync_all**, every connected domain agrees on the new state.

theorem *cross-domain-consistency*:

```

assumes source  $\in$  domains
and domain-state source aid = Some s
and s  $\in$  states
and action  $\in$  actions
and transition s action = Some s'
and sync-all source action aid domain-state = Some ds'
and d  $\in$  connected-domains aid
shows ds' d aid = Some s'
proof –
from assms(2,5,6) have
  ds' = ( $\lambda d$  aid'.
    if aid' = aid  $\wedge d \in$  connected-domains aid
    then Some s'
    else domain-state d aid')
unfolding sync-all-def by auto
with assms(7) show ?thesis by simp
qed

```

sync_all does not affect other assets.

theorem *sync-isolation*:

```

assumes sync-all source action aid domain-state = Some ds'
and aid'  $\neq$  aid
shows ds' d aid' = domain-state d aid'
proof –
from assms(1) obtain s s' where
  domain-state source aid = Some s
  transition s action = Some s'
unfolding sync-all-def by (auto split: option.splits)
then have ds' = ( $\lambda d$  aid'.
  if aid' = aid  $\wedge d \in$  connected-domains aid
  then Some s'
  else domain-state d aid')
using assms(1) unfolding sync-all-def by auto

```

```

    with assms(2) show ?thesis by auto
qed

end

end

```

```

theory Regulatory-Instance
  imports State-Preservation
begin

```

6 Regulatory State and Action Definitions

```

datatype reg-state = ACTIVE | FROZEN | SEIZED | CONFISCATED | RE-
STRICTED

```

```

datatype reg-action = FREEZE | SEIZE | CONFISCATE | RESTRICT
| UNFREEZE | UNRESTRICT | RELEASE

```

The regulatory transition function. Partial: returns None for invalid (state, action) combinations. The transition table encodes legal precedence: seizure is stronger than freezing, confiscation is terminal and universally reachable, and intermediate states must return to Active before lateral transitions.

```

fun reg-transition :: reg-state  $\Rightarrow$  reg-action  $\Rightarrow$  reg-state option where
  — From ACTIVE: all forward actions valid
  | reg-transition ACTIVE FREEZE      = Some FROZEN
  | reg-transition ACTIVE SEIZE       = Some SEIZED
  | reg-transition ACTIVE CONFISCATE = Some CONFISCATED
  | reg-transition ACTIVE RESTRICT   = Some RESTRICTED
  | reg-transition ACTIVE UNFREEZE   = None
  | reg-transition ACTIVE UNRESTRICT = None
  | reg-transition ACTIVE RELEASE    = None
  — From FROZEN: unfreeze, escalate to SEIZED or CONFISCATED
  | reg-transition FROZEN UNFREEZE   = Some ACTIVE
  | reg-transition FROZEN SEIZE       = Some SEIZED
  | reg-transition FROZEN CONFISCATE = Some CONFISCATED
  | reg-transition FROZEN FREEZE     = None
  | reg-transition FROZEN RESTRICT   = None
  | reg-transition FROZEN UNRESTRICT = None
  | reg-transition FROZEN RELEASE    = None
  — From SEIZED: release or final confiscation only
  | reg-transition SEIZED RELEASE    = Some ACTIVE
  | reg-transition SEIZED CONFISCATE = Some CONFISCATED
  | reg-transition SEIZED FREEZE     = None
  | reg-transition SEIZED SEIZE      = None
  | reg-transition SEIZED RESTRICT   = None

```

```

| reg-transition SEIZED UNFREEZE    = None
| reg-transition SEIZED UNRESTRICT = None
| — From CONFISCATED: terminal no transitions out
| reg-transition CONFISCATED FREEZE    = None
| reg-transition CONFISCATED SEIZE     = None
| reg-transition CONFISCATED CONFISCATE = None
| reg-transition CONFISCATED RESTRICT  = None
| reg-transition CONFISCATED UNFREEZE  = None
| reg-transition CONFISCATED UNRESTRICT = None
| reg-transition CONFISCATED RELEASE  = None
| — From RESTRICTED: unrestrict, escalate to FROZEN or CONFISCATED
| reg-transition RESTRICTED UNRESTRICT = Some ACTIVE
| reg-transition RESTRICTED FREEZE     = Some FROZEN
| reg-transition RESTRICTED CONFISCATE = Some CONFISCATED
| reg-transition RESTRICTED RESTRICT  = None
| reg-transition RESTRICTED SEIZE     = None
| reg-transition RESTRICTED UNFREEZE  = None
| reg-transition RESTRICTED RELEASE  = None

```

7 Fundamental Properties of the Transition Function

7.1 Invariant I1: CONFISCATED is terminal

lemma *confiscated-terminal*:
 reg-transition CONFISCATED a = None
 by (cases a) auto

7.2 Invariant I2: CONFISCATE reaches CONFISCATED from any non-terminal state

lemma *confiscate-universal*:
 $s \neq \text{CONFISCATED} \implies \text{reg-transition } s \text{ CONFISCATE} = \text{Some CONFISCATED}$
 by (cases s) auto

7.3 Invariant I3: Determinism (inherent from function definition)

Determinism is trivially guaranteed by the **fun** definition.

7.4 SEIZED to FROZEN exclusion

SEIZED is a strictly stronger constraint than FROZEN. Direct transition from SEIZED to FROZEN is legally nonsensical (cannot "weaken" a court-ordered seizure to a mere freeze). The path is: RELEASE then ACTIVE then FREEZE.

lemma *seized-no-freeze*:
reg-transition SEIZED FREEZE = None
by *simp*

lemma *seized-no-unfreeze*:
reg-transition SEIZED UNFREEZE = None
by *simp*

lemma *seized-no-restrict*:
reg-transition SEIZED RESTRICT = None
by *simp*

7.5 FROZEN to RESTRICTED exclusion

Must go through ACTIVE: UNFREEZE then ACTIVE then RESTRICT.

lemma *frozen-no-restrict*:
reg-transition FROZEN RESTRICT = None
by *simp*

7.6 Transition completeness: every non-terminal state has at least one valid action

lemma *non-terminal-has-action*:
assumes $s \neq \text{CONFISCATED}$
shows $\exists a. \text{reg-transition } s \ a \neq \text{None}$
proof (*cases* s)
 case *ACTIVE*
 show *?thesis*
 apply (*rule* *exI*[*where* $x = \text{FREEZE}$])
 using *ACTIVE* **by** *simp*
next
 case *FROZEN*
 show *?thesis*
 apply (*rule* *exI*[*where* $x = \text{UNFREEZE}$])
 using *FROZEN* **by** *simp*
next
 case *SEIZED*
 show *?thesis*
 apply (*rule* *exI*[*where* $x = \text{RELEASE}$])
 using *SEIZED* **by** *simp*
next
 case *CONFISCATED*
 then show *?thesis* **using** *assms* **by** *contradiction*
next
 case *RESTRICTED*
 show *?thesis*
 apply (*rule* *exI*[*where* $x = \text{UNRESTRICT}$])
 using *RESTRICTED* **by** *simp*
qed

7.7 Reachability from ACTIVE

Every state is reachable from ACTIVE via some sequence of actions.

lemma *active-reaches-frozen*: *reg-transition ACTIVE FREEZE = Some FROZEN*
by *simp*

lemma *active-reaches-seized*: *reg-transition ACTIVE SEIZE = Some SEIZED*
by *simp*

lemma *active-reaches-confiscated*: *reg-transition ACTIVE CONFISCATE = Some CONFISCATED*
by *simp*

lemma *active-reaches-restricted*: *reg-transition ACTIVE RESTRICT = Some RESTRICTED*
by *simp*

7.8 Return paths to ACTIVE

lemma *frozen-returns*: *reg-transition FROZEN UNFREEZE = Some ACTIVE*
by *simp*

lemma *seized-returns*: *reg-transition SEIZED RELEASE = Some ACTIVE*
by *simp*

lemma *restricted-returns*: *reg-transition RESTRICTED UNRESTRICT = Some ACTIVE*
by *simp*

8 State Machine Instance

We show that `reg_state` / `reg_action` / `reg_transition` satisfy the `state_machine` locale assumptions.

definition *reg-states* :: *reg-state set* **where**
reg-states = {*ACTIVE*, *FROZEN*, *SEIZED*, *CONFISCATED*, *RESTRICTED*}

definition *reg-actions* :: *reg-action set* **where**
reg-actions = {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*, *UNFREEZE*, *UNRESTRICT*, *RELEASE*}

definition *reg-terminal* :: *reg-state set* **where**
reg-terminal = {*CONFISCATED*}

lemma *reg-states-UNIV*: *reg-states* = *UNIV*
unfolding *reg-states-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *reg-actions-UNIV*: *reg-actions* = *UNIV*
unfolding *reg-actions-def* **by** (*auto*, *case-tac x*, *auto*)

```

lemma reg-transition-closed:
   $\llbracket \text{reg-transition } s \ a = \text{Some } s' \rrbracket \implies s' \in \text{reg-states}$ 
  unfolding reg-states-def by (cases s; cases a; auto)

interpretation reg-sm: state-machine reg-states reg-actions reg-transition reg-terminal
proof unfold-locales
  show finite reg-states unfolding reg-states-def by auto
next
  show finite reg-actions unfolding reg-actions-def by auto
next
  show  $\text{reg-terminal} \subseteq \text{reg-states}$  unfolding reg-terminal-def reg-states-def by auto
next
  fix s a
  assume  $s \in \text{reg-terminal} \ a \in \text{reg-actions}$ 
  then show  $\text{reg-transition } s \ a = \text{None}$ 
    unfolding reg-terminal-def by (auto simp: confiscated-terminal)
next
  fix s a s'
  assume  $s \in \text{reg-states} \ a \in \text{reg-actions} \ \text{reg-transition } s \ a = \text{Some } s'$ 
  then show  $s' \in \text{reg-states}$ 
    using reg-transition-closed by auto
next
  fix s :: reg-state and a :: reg-action
  assume  $s \notin \text{reg-states}$ 
  then show  $\text{reg-transition } s \ a = \text{None}$ 
    using reg-states-UNIV by auto
qed

```

9 Asset and Global State Modeling

```

type-synonym asset-id = nat
type-synonym chain-id = nat
type-synonym authority-id = nat
type-synonym timestamp = nat

record asset-state =
  as-asset-id :: asset-id
  as-reg-state :: reg-state
  as-owner    :: nat      — Abstract address
  as-locked   :: bool     — Preemptive lock

type-synonym chain-state = asset-id  $\Rightarrow$  asset-state option

record global-state =
  gs-chains    :: chain-id  $\Rightarrow$  chain-state
  gs-locks     :: asset-id  $\Rightarrow$  bool

record oss-message =

```

msg-action :: *reg-action*
msg-asset-id :: *asset-id*
msg-authority :: *authority-id*
msg-timestamp :: *timestamp*
msg-source :: *chain-id*
msg-targets :: *chain-id set*

10 State Accessors and Predicates

definition *get-asset-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *asset-state option* **where**

get-asset-state *gs* *cid* *aid* = *gs-chains* *gs* *cid* *aid*

definition *get-reg-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state option* **where**

get-reg-state *gs* *cid* *aid* =
 (case *get-asset-state* *gs* *cid* *aid* of
 None $\Rightarrow$ *None*
 | *Some ast* $\Rightarrow$ *Some* (*as-reg-state* *ast*))

definition *asset-exists* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *bool* **where**

asset-exists *gs* *cid* *aid* = (*get-asset-state* *gs* *cid* *aid* $\neq$ *None*)

definition *is-locked* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *bool* **where**

is-locked *gs* *aid* = *gs-locks* *gs* *aid*

Connected chains: all chains holding a given asset.

definition *connected-chains* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *chain-id set* **where**

connected-chains *gs* *aid* = {*cid*. *asset-exists* *gs* *cid* *aid*}

11 Global State Validity

A global state is valid if all chains holding the same asset agree on its regulatory state. This is the cross-chain consistency invariant that synchronization must preserve.

definition *consistent-state* :: *global-state* $\Rightarrow$ *bool* **where**

consistent-state *gs* $\equiv$
 $\forall$ *c1* *c2* *aid* *s1* *s2*.
get-reg-state *gs* *c1* *aid* = *Some* *s1* $\longrightarrow$
get-reg-state *gs* *c2* *aid* = *Some* *s2* $\longrightarrow$
s1 = *s2*

definition *no-locked-without-reason* :: *global-state* $\Rightarrow$ *bool* **where**

no-locked-without-reason *gs* $\equiv$
 $\forall$ *aid*. $\neg$ *is-locked* *gs* *aid*

— In a quiescent valid state, no assets are locked. Locks are transient, held only during sync operations.

definition *valid-state* :: *global-state* $\Rightarrow$ *bool* **where**
valid-state *gs* $\equiv$ *consistent-state* *gs* $\wedge$ *no-locked-without-reason* *gs*

12 Synchronization Protocol

12.1 Lock acquisition

definition *acquire-lock* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state option* **where**
acquire-lock *gs* *aid* =
 (if *is-locked* *gs* *aid* then *None*
 else *Some* (*gs* $\setminus$ *gs-locks* := (*gs-locks* *gs*)(*aid* := *True*) $\setminus$)))

definition *release-lock* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state* **where**
release-lock *gs* *aid* = *gs* $\setminus$ *gs-locks* := (*gs-locks* *gs*)(*aid* := *False*) $\setminus$

12.2 State update across chains

Update the regulatory state of an asset on a specific chain. Preserves all other asset fields (owner, etc.).

definition *update-chain-reg-state* ::
global-state $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state* $\Rightarrow$ *global-state*
where
update-chain-reg-state *gs* *cid* *aid* *new-st* =
 (let *old-chain* = *gs-chains* *gs* *cid* in
 let *new-chain* = (λ *aid'*.
 if *aid'* = *aid* then
 (case *old-chain* *aid* of
 None $\Rightarrow$ None
 | *Some* *ast* $\Rightarrow$ *Some* (*ast* $\setminus$ *as-reg-state* := *new-st*) $\setminus$))
 else *old-chain* *aid'*)
 in *gs* $\setminus$ *gs-chains* := (*gs-chains* *gs*)(*cid* := *new-chain*) $\setminus$))

Update all connected chains to a new regulatory state.

We define this directly rather than using `Finite_Set.fold`, because the update for a given asset on each chain is independent we simply construct a new global state where every chain's view of the asset is updated. This avoids the need for `comp_fun_commute` proofs.

definition *update-all-chains* ::
global-state $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state* $\Rightarrow$ *chain-id set* $\Rightarrow$ *global-state*
where
update-all-chains *gs* *aid* *new-st* *targets* =
gs $\setminus$ *gs-chains* := (λ *cid*.
 if *cid* $\in$ *targets* then
 (λ *aid'*. if *aid'* = *aid* then
 (case *gs-chains* *gs* *cid* *aid* of
 None $\Rightarrow$ None
 | *Some* *ast* $\Rightarrow$ *Some* (*ast* $\setminus$ *as-reg-state* := *new-st*) $\setminus$))
 else *gs-chains* *gs* *cid* *aid* $\setminus$))

```

    else gs-chains gs cid aid')
  else gs-chains gs cid)  $\mathbb{I}$ 

```

12.3 The synchronization function

sync: the core synchronization operation.

Given a source chain, action, and **asset_id**: 1. Verify the asset exists on the source chain 2. Check the transition is valid 3. Acquire global lock (fails if already locked) 4. Update all connected chains to the new state 5. Release lock

Returns *None* if any step fails (invalid transition, lock contention, etc.)

definition *sync* ::

chain-id $\Rightarrow$ *reg-action* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state* $\Rightarrow$ *global-state option*

where

```

sync source action aid gs =
  (case get-reg-state gs source aid of
    None  $\Rightarrow$  None
  | Some current-st  $\Rightarrow$ 
    (case reg-transition current-st action of
      None  $\Rightarrow$  None
    | Some new-st  $\Rightarrow$ 
      (case acquire-lock gs aid of
        None  $\Rightarrow$  None
      | Some gs-locked  $\Rightarrow$ 
        let targets = connected-chains gs aid in
        let gs-updated = update-all-chains gs-locked aid new-st targets in
        Some (release-lock gs-updated aid))))

```

13 Synchronization Properties

13.1 Idempotency

After synchronization, applying the same action again yields *None* (the state has already transitioned, so the same transition from the new state is typically invalid) or is a no-op.

More precisely: if *sync* succeeds and produces **new_st**, then **reg_transition new_st action** = *None* for most action/state pairs (the transition table has no self-loops).

lemma *no-self-loops*:

```

reg-transition s a = Some s  $\implies$  False
by (cases s; cases a; auto)

```

lemma *transition-changes-state*:

```

reg-transition s a = Some s'  $\implies$  s  $\neq$  s'
using no-self-loops by auto

```

13.2 Sync isolation: other assets are not affected

Synchronization on asset *aid* does not change the state of any other asset (where the asset identifier differs) on any chain.

```
lemma update-chain-other-asset:  
  assumes  $aid' \neq aid$   
  shows  $gs-chains (update-chain-reg-state\ gs\ cid\ aid\ new-st)\ cid'\ aid' =$   
     $gs-chains\ gs\ cid'\ aid'$   
proof (cases  $cid' = cid$ )  
  case True  
    then show ?thesis  
    unfolding update-chain-reg-state-def Let-def  
    using assms by auto  
next  
  case False  
    then show ?thesis  
    unfolding update-chain-reg-state-def Let-def by auto  
qed
```

13.3 Sync correctness: source chain updated

After *update_chain_reg_state*, the target chain reflects the new state.

```
lemma update-chain-same-asset:  
  assumes  $gs-chains\ gs\ cid\ aid = Some\ ast$   
  shows  $gs-chains (update-chain-reg-state\ gs\ cid\ aid\ new-st)\ cid\ aid =$   
     $Some\ (ast \lfloor as-reg-state := new-st \rfloor)$   
  unfolding update-chain-reg-state-def Let-def  
  using assms by auto
```

13.4 Properties of *update_all_chains*

update_all_chains correctly updates any chain in the target set.

```
lemma update-all-chains-in-targets:  
  assumes  $cid \in targets$   
  and  $gs-chains\ gs\ cid\ aid = Some\ ast$   
  shows  $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid =$   
     $Some\ (ast \lfloor as-reg-state := new-st \rfloor)$   
  unfolding update-all-chains-def using assms by auto
```

```
lemma update-all-chains-in-targets-none:  
  assumes  $cid \in targets$   
  and  $gs-chains\ gs\ cid\ aid = None$   
  shows  $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid = None$   
  unfolding update-all-chains-def using assms by auto
```

```
lemma update-all-chains-outside-targets:  
  assumes  $cid \notin targets$   
  shows  $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid = gs-chains\ gs\ cid$ 
```

unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-other-asset*:

assumes $aid' \neq aid$

shows $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid' =$
 $gs-chains\ gs\ cid\ aid'$

unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-locks*:

$gs-locks (update-all-chains\ gs\ aid\ new-st\ targets) = gs-locks\ gs$

unfolding *update-all-chains-def* **by** *auto*

Key lemma: `get_reg_state` after `update_all_chains` for a chain in targets.

lemma *update-all-chains-reg-state*:

assumes $cid \in targets$

and $get-asset-state\ gs\ cid\ aid = Some\ ast$

shows $get-reg-state (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid = Some\ new-st$

proof –

from *assms*(2) **have** $chain: gs-chains\ gs\ cid\ aid = Some\ ast$

unfolding *get-asset-state-def* **by** *simp*

then have $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid =$
 $Some (ast \lfloor as-reg-state := new-st \rfloor)$

using *update-all-chains-in-targets[OF assms(1)]* **by** *simp*

then show *?thesis*

unfolding *get-reg-state-def* *get-asset-state-def* **by** *simp*

qed

13.5 Lock mutual exclusion

If a lock is held, concurrent sync on the same asset fails. This prevents concurrent regulatory action conflicts.

lemma *lock-mutual-exclusion*:

assumes $is-locked\ gs\ aid$

shows $acquire-lock\ gs\ aid = None$

unfolding *acquire-lock-def* **using** *assms* **by** *auto*

lemma *lock-acquire-success*:

assumes $\neg is-locked\ gs\ aid$

shows $\exists gs'. acquire-lock\ gs\ aid = Some\ gs' \wedge is-locked\ gs'\ aid$

proof –

from *assms* **have** $\neg gs-locks\ gs\ aid$

unfolding *is-locked-def* **by** *simp*

then have $acquire-lock\ gs\ aid = Some (gs \lfloor gs-locks := (gs-locks\ gs)(aid := True) \rfloor)$

$\rfloor)$

unfolding *acquire-lock-def* *is-locked-def* **by** *simp*

moreover have $is-locked (gs \lfloor gs-locks := (gs-locks\ gs)(aid := True) \rfloor) aid$

unfolding *is-locked-def* **by** *simp*

ultimately show *?thesis* by auto
qed

lemma *lock-release*:
shows $\neg$ *is-locked* (*release-lock* *gs* *aid*) *aid*
unfolding *release-lock-def is-locked-def* by auto

14 Main Theorem: Cross-Chain Regulatory Consistency

The central theorem of this theory. Under a valid global state (all chains consistent, no outstanding locks), if a regulatory action is applied via sync, the resulting state is also valid (all chains consistent) and every connected chain reflects the new regulatory state.

This is the regulatory instantiation of the cross-domain consistency theorem from **State_Preservation.thy**.

The proof proceeds by: 1. Unfolding sync into lock, then **update_all_chains**, then unlock 2. Showing **acquire_lock** and **release_lock** preserve chain state 3. Showing **update_all_chains** updates every chain in the target set 4. Combining these facts to establish the conclusion

Auxiliary: update on a single chain preserves consistency for other assets.

lemma *update-chain-preserves-other-consistency*:
assumes *consistent-state* *gs*
and *aid'* $\neq$ *aid*
and *get-reg-state* *gs* *c1* *aid'* = *Some s1*
and *get-reg-state* *gs* *c2* *aid'* = *Some s2*
shows *get-reg-state* (*update-chain-reg-state* *gs* *cid* *aid* *new-st*) *c1* *aid'* = *Some s1*
 $\wedge$ *get-reg-state* (*update-chain-reg-state* *gs* *cid* *aid* *new-st*) *c2* *aid'* = *Some s2*
using *assms* unfolding *get-reg-state-def get-asset-state-def*
update-chain-reg-state-def Let-def consistent-state-def
by auto

Auxiliary: **release_lock** does not change chains.

lemma *release-lock-chains*:
gs-chains (*release-lock* *gs* *aid*) = *gs-chains* *gs*
unfolding *release-lock-def* by *simp*

lemma *release-lock-reg-state*:
get-reg-state (*release-lock* *gs* *aid*) *cid* *aid'* = *get-reg-state* *gs* *cid* *aid'*
unfolding *get-reg-state-def get-asset-state-def release-lock-chains* by *simp*

Auxiliary: **acquire_lock** does not change chains.

lemma *acquire-lock-chains*:
assumes *acquire-lock* *gs* *aid* = *Some gs'*
shows *gs-chains* *gs'* = *gs-chains* *gs*
using *assms* unfolding *acquire-lock-def is-locked-def*

by (*auto split: if-splits*)

lemma *acquire-lock-reg-state*:

assumes *acquire-lock* *gs aid = Some gs'*
shows *get-reg-state* *gs' cid aid' = get-reg-state* *gs cid aid'*
using *acquire-lock-chains*[*OF* *assms*]
unfolding *get-reg-state-def* *get-asset-state-def* **by** *simp*

Auxiliary: **connected_chains** uses original *gs*, not locked *gs*.

lemma *acquire-lock-asset-exists*:

assumes *acquire-lock* *gs aid = Some gs'*
shows *asset-exists* *gs' cid aid' = asset-exists* *gs cid aid'*
using *acquire-lock-chains*[*OF* *assms*]
unfolding *asset-exists-def* *get-asset-state-def* **by** *simp*

The cross-chain regulatory homomorphism theorem.

theorem *regulatory-homomorphism*:

assumes *valid: valid-state* *gs*
and *exists: asset-exists* *gs source aid*
and *current: get-reg-state* *gs source aid = Some s*
and *trans: reg-transition* *s action = Some s'*
and *synced: sync* *source action aid gs = Some gs'*
and *connected: c ∈ connected-chains* *gs aid*
and *fin: finite* (*connected-chains* *gs aid*)
shows *get-reg-state* *gs' c aid = Some s'*

proof —

— Step 1: **Unfold** *sync* to extract intermediate states

from *synced current trans*

obtain *gs-locked* **where**

lock: acquire-lock *gs aid = Some gs-locked*

and *gs'-def: gs' = release-lock*

(*update-all-chains* *gs-locked aid s' (connected-chains* *gs aid)*) *aid*

unfolding *sync-def*

by (*auto split: option.splits simp: Let-def*)

— Step 2: **acquire_lock** preserves chains

have *locked-chains: gs-chains* *gs-locked = gs-chains* *gs*

using *acquire-lock-chains*[*OF* *lock*] **by** *simp*

— Step 3: *c* is in **connected_chains**, so the asset exists on *c*

from *connected* **have** *asset-exists* *gs c aid*

unfolding *connected-chains-def* **by** *simp*

then obtain *ast-c* **where** *ast-c: gs-chains* *gs c aid = Some ast-c*

unfolding *asset-exists-def* *get-asset-state-def* **by** *auto*

— Step 4: Therefore asset also exists in **gs_locked** on chain *c*

have *ast-locked: gs-chains* *gs-locked c aid = Some ast-c*

using *ast-c locked-chains* **by** *simp*

— Step 5: `update_all_chains` updates chain `c`
have *updated*: *gs-chains* (*update-all-chains* *gs-locked* *aid* *s'* (*connected-chains* *gs* *aid*)) *c* *aid* =
 Some (*ast-c* | *as-reg-state* := *s'* |)
using *update-all-chains-in-targets*[*OF* *connected* *ast-locked*] **by** *simp*

— Step 6: `release_lock` does not change chains
have *gs-chains* *gs'* *c* *aid* = *Some* (*ast-c* | *as-reg-state* := *s'* |)
using *updated* **unfolding** *gs'-def* *release-lock-chains* **by** *simp*

— Step 7: Extract `reg_state`
then show *?thesis*
unfolding *get-reg-state-def* *get-asset-state-def* **by** *simp*
qed

15 Valid State Preservation

The central safety property: synchronization preserves global state validity.

If the global state is valid (consistent and unlocked) before sync, and sync succeeds, then the resulting global state is also valid.

This closes the inductive invariant: valid initial state + `valid_state` preservation under `sync` = `valid_state` holds at all reachable states.

The proof decomposes into two parts: (1) `consistent_state` preservation: sync updates all connected chains to the same new state and leaves other assets untouched. (2) `no_locked_without_reason` preservation: sync acquires and then releases the lock within the same operation, so the output has no outstanding locks (assuming no other locks were held).

15.1 Part 1: `consistent_state` preservation

After sync, any two chains holding asset *aid* agree on its new regulatory state (it is *s'*), and any two chains holding a different asset *aid'* still agree (their states are unchanged from the consistent input).

lemma *sync-preserves-consistent-state*:

assumes *valid*: *valid-state* *gs*
and *exists*: *asset-exists* *gs* *source* *aid*
and *current*: *get-reg-state* *gs* *source* *aid* = *Some* *s*
and *trans*: *reg-transition* *s* *action* = *Some* *s'*
and *synced*: *sync* *source* *action* *aid* *gs* = *Some* *gs'*
and *fin*: *finite* (*connected-chains* *gs* *aid*)
shows *consistent-state* *gs'*

proof —

— Unfold sync to extract the structure
from *synced* *current* *trans*
obtain *gs-locked* **where**
 lock: *acquire-lock* *gs* *aid* = *Some* *gs-locked*

```

and gs'-def: gs' = release-lock
      (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
unfolding sync-def
by (auto split: option.splits simp: Let-def)

have locked-chains: gs-chains gs-locked = gs-chains gs
using acquire-lock-chains[OF lock] by simp

show ?thesis
unfolding consistent-state-def
proof (intro allI impI)
  fix c1 c2 aid' s1-final s2-final
  assume s1-eq: get-reg-state gs' c1 aid' = Some s1-final
    and s2-eq: get-reg-state gs' c2 aid' = Some s2-final

  show s1-final = s2-final
  proof (cases aid' = aid)
    case True
    — Case: the synchronized asset. Two sub-cases per chain.
    show ?thesis
    proof (cases c1 ∈ connected-chains gs aid)
      case c1-in: True
      then have ae1: asset-exists gs c1 aid
        unfolding connected-chains-def by simp
      then obtain ast1 where ast1: gs-chains gs c1 aid = Some ast1
        unfolding asset-exists-def get-asset-state-def by auto
      have ast1-locked: gs-chains gs-locked c1 aid = Some ast1
        using ast1 locked-chains by simp
      have upd1: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c1 aid =
        Some (ast1() as-reg-state := s')
        using update-all-chains-in-targets[OF c1-in ast1-locked] by simp
      have reg1: get-reg-state gs' c1 aid = Some s'
        unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
        using upd1 by simp
      show ?thesis
      proof (cases c2 ∈ connected-chains gs aid)
        case c2-in: True
        then have ae2: asset-exists gs c2 aid
          unfolding connected-chains-def by simp
        then obtain ast2 where ast2: gs-chains gs c2 aid = Some ast2
          unfolding asset-exists-def get-asset-state-def by auto
        have ast2-locked: gs-chains gs-locked c2 aid = Some ast2
          using ast2 locked-chains by simp
        have reg2: get-reg-state gs' c2 aid = Some s'
          unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
          using update-all-chains-in-targets[OF c2-in ast2-locked] by simp
        from reg1 reg2 s1-eq s2-eq True show ?thesis by simp
      next

```

```

case c2-out: False
  — c2 is not connected, so asset doesn't exist on c2 for aid
  have gs-chains (update-all-chains gs-locked aid s' (connected-chains gs
aid)) c2 =
    gs-chains gs-locked c2
    using update-all-chains-outside-targets[OF c2-out] by simp
    then have gs-chains gs' c2 aid = gs-chains gs c2 aid
    unfolding gs'-def release-lock-chains using locked-chains by simp
    moreover have gs-chains gs c2 aid = None
    using c2-out unfolding connected-chains-def asset-exists-def get-asset-state-def
    by auto
    ultimately have get-reg-state gs' c2 aid = None
    unfolding get-reg-state-def get-asset-state-def by simp
    with s2-eq True show ?thesis by simp
qed
next
case c1-out: False
  — c1 not connected: asset doesn't exist there
  have gs-chains (update-all-chains gs-locked aid s' (connected-chains gs aid))
c1 =
    gs-chains gs-locked c1
    using update-all-chains-outside-targets[OF c1-out] by simp
    then have gs-chains gs' c1 aid = gs-chains gs c1 aid
    unfolding gs'-def release-lock-chains using locked-chains by simp
    moreover have gs-chains gs c1 aid = None
    using c1-out unfolding connected-chains-def asset-exists-def get-asset-state-def
    by auto
    ultimately have get-reg-state gs' c1 aid = None
    unfolding get-reg-state-def get-asset-state-def by simp
    with s1-eq True show ?thesis by simp
qed
next
case diff: False
  — Case: a different asset. Sync does not touch it.
  have other1: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c1 aid' =
    gs-chains gs-locked c1 aid'
    using update-all-chains-other-asset[OF diff] by simp
    have other2: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c2 aid' =
    gs-chains gs-locked c2 aid'
    using update-all-chains-other-asset[OF diff] by simp
    have gs-chains gs' c1 aid' = gs-chains gs c1 aid'
    unfolding gs'-def release-lock-chains using other1 locked-chains by simp
    moreover have gs-chains gs' c2 aid' = gs-chains gs c2 aid'
    unfolding gs'-def release-lock-chains using other2 locked-chains by simp
    ultimately have get-reg-state gs' c1 aid' = get-reg-state gs c1 aid'
    and get-reg-state gs' c2 aid' = get-reg-state gs c2 aid'
    unfolding get-reg-state-def get-asset-state-def by simp-all

```

```

with s1-eq s2-eq have
  get-reg-state gs c1 aid' = Some s1-final
  get-reg-state gs c2 aid' = Some s2-final
  by simp-all
with valid show ?thesis
  unfolding valid-state-def consistent-state-def by blast
qed
qed
qed

```

15.2 Part 2: no_locked_without_reason preservation

After sync, no assets are locked. The sync operation acquires a lock on the target asset and releases it before returning. For all other assets, the lock state is unchanged (and was False by `valid_state`).

lemma *sync-preserves-no-locks:*

```

assumes valid: valid-state gs
and exists: asset-exists gs source aid
and current: get-reg-state gs source aid = Some s
and trans: reg-transition s action = Some s'
and synced: sync source action aid gs = Some gs'
shows no-locked-without-reason gs'
proof —
from synced current trans
obtain gs-locked where
  lock: acquire-lock gs aid = Some gs-locked
and gs'-def: gs' = release-lock
  (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
unfolding sync-def
by (auto split: option.splits simp: Let-def)

```

— Locks after acquire: only aid is locked

```

from lock have locked-locks: gs-locks gs-locked = (gs-locks gs)(aid := True)
unfolding acquire-lock-def is-locked-def
by (auto split: if-splits)

```

— `update_all_chains` does not change locks

```

have upd-locks: gs-locks (update-all-chains gs-locked aid s' (connected-chains gs aid)) =
  gs-locks gs-locked
using update-all-chains-locks by simp

```

— `release_lock` clears the lock on aid

```

have final-locks: gs-locks gs' = (gs-locks gs-locked)(aid := False)
unfolding gs'-def release-lock-def using upd-locks by simp

```

show *?thesis*

unfolding *no-locked-without-reason-def is-locked-def*

proof

```

fix aid'
show  $\neg$  gs-locks gs' aid'
proof (cases aid' = aid)
  case True
  then show ?thesis using final-locks by simp
next
case False
then have gs-locks gs' aid' = gs-locks gs-locked aid'
  using final-locks by simp
also have ... = gs-locks gs aid'
  using locked-locks False by simp
also have ... = False
using valid unfolding valid-state-def no-locked-without-reason-def is-locked-def
  by simp
finally show ?thesis by simp
qed
qed
qed

```

15.3 The valid_state preservation theorem

Combining the two parts: sync preserves **valid_state**. This is the inductive invariant that guarantees **valid_state** holds at every reachable global state.

theorem *valid-state-preservation:*

```

assumes valid: valid-state gs
  and exists: asset-exists gs source aid
  and current: get-reg-state gs source aid = Some s
  and trans: reg-transition s action = Some s'
  and synced: sync source action aid gs = Some gs'
  and fin: finite (connected-chains gs aid)
shows valid-state gs'
unfolding valid-state-def
using sync-preserves-consistent-state[OF assms]
  sync-preserves-no-locks[OF valid exists current trans synced]
by auto

```

16 Multi-Domain Locale Instantiation

We now instantiate the **multi_domain_preservation** locale from **State_Preservation.thy** with the regulatory state machine.

The instantiation is parametric: given ANY finite set of domains and ANY initial **domain_state** function satisfying the consistency precondition, the locale is satisfied. This proves that the abstract framework applies to the concrete regulatory model without fixing a specific topology.

The key bridge: we construct a **domain_state** function from our **global_state**, and show that **valid_state** implies the consistency precondition required by **multi_domain_preservation**.

Bridge from our `global_state` model to the `multi_domain_preservation` locale's `domain_state` signature.

`multi_domain_preservation` expects: `domain_state :: 'd => 'id => 's option`

We instantiate with: `'d = chain_id, 'id = asset_id, 's = reg_state`

The bridge function extracts `reg_state` from our richer `asset_state`.

definition *reg-domain-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state option* **where**

reg-domain-state *gs cid aid* = *get-reg-state* *gs cid aid*

The parametric instantiation theorem: for any finite set of `chain_ids` and any valid `global_state`, we can interpret the `multi_domain_preservation` locale with the regulatory state machine.

The `state_machine` predicate as a standalone fact. This is needed for instantiating `multi_domain_preservation`, which takes `state_machine` as an opaque predicate assumption.

lemma *reg-state-machine-pred*:

state-machine *reg-states* *reg-actions* *reg-transition* *reg-terminal*

proof –

have *f1*: *finite* *reg-states* **unfolding** *reg-states-def* **by** *auto*

have *f2*: *finite* *reg-actions* **unfolding** *reg-actions-def* **by** *auto*

have *f3*: *reg-terminal* $\subseteq$ *reg-states*

unfolding *reg-terminal-def* *reg-states-def* **by** *auto*

have *f4*: $\bigwedge s \ a. s \in \text{reg-terminal} \implies a \in \text{reg-actions} \implies \text{reg-transition } s \ a = \text{None}$

unfolding *reg-terminal-def* **by** (*auto simp: confiscated-terminal*)

have *f5*: $\bigwedge s \ a \ s'. s \in \text{reg-states} \implies a \in \text{reg-actions} \implies \text{reg-transition } s \ a = \text{Some } s' \implies s' \in \text{reg-states}$

using *reg-transition-closed* **by** *auto*

have *f6*: $\bigwedge s \ a. (s :: \text{reg-state}) \notin \text{reg-states} \implies \text{reg-transition } s \ a = \text{None}$

using *reg-states-UNIV* **by** *auto*

show *?thesis*

by (*intro state-machine.intro*) (*use f1 f2 f3 f4 f5 f6 in auto*)

qed

theorem *reg-multi-domain-instantiation*:

assumes *fin-doms*: *finite* (*doms* :: *chain-id* *set*)

and *valid*: *valid-state* *gs*

shows *multi-domain-preservation* *doms* *reg-states* *reg-actions* *reg-transition* *reg-terminal* (*reg-domain-state* *gs*)

proof (*intro multi-domain-preservation.intro*)

show *finite* *doms* **using** *fin-doms* **by** *simp*

next

show *state-machine* *reg-states* *reg-actions* *reg-transition* *reg-terminal*

by (*rule reg-state-machine-pred*)

next

fix *d1 d2* :: *chain-id* **and** *aid* :: *asset-id* **and** *s1 s2* :: *reg-state*

```

assume  $d1 \in \text{doms}$   $d2 \in \text{doms}$ 
and  $\text{reg-domain-state } gs \ d1 \ \text{aid} = \text{Some } s1$ 
and  $\text{reg-domain-state } gs \ d2 \ \text{aid} = \text{Some } s2$ 
then have  $\text{get-reg-state } gs \ d1 \ \text{aid} = \text{Some } s1$ 
and  $\text{get-reg-state } gs \ d2 \ \text{aid} = \text{Some } s2$ 
unfolding  $\text{reg-domain-state-def}$  by  $\text{simp-all}$ 
with  $\text{valid}$  show  $s1 = s2$ 
unfolding  $\text{valid-state-def}$   $\text{consistent-state-def}$  by  $\text{blast}$ 
qed

```

Corollary: the generic `cross_domain_consistency` theorem from `State_Preservation.thy` applies to regulatory synchronization.

This connects the abstract theory to the concrete model: the generic theorem guarantees that after `sync_all` (the abstract synchronization), all connected domains agree. Combined with our concrete `regulatory_homomorphism` theorem, we have both the abstract and concrete guarantees.

The instantiation above (`reg_multi_domain_instantiation`) enables using all theorems proven in `multi_domain_preservation` in particular `cross_domain_consistency` and `sync_isolation` for the regulatory model without reproving them. Any consumer of this theory can invoke:

```

multi_domain_preservation.cross_domain_consistency [OF reg_multi_domain_instantiation
fin_doms valid_gs]]

```

to obtain the concrete guarantee for their domain set and global state.

Summary of what this theory establishes:

1. The regulatory state machine (`reg_state`, `reg_action`, `reg_transition`) satisfies the `state_machine` locale from `State_Preservation.thy`.

2. CONFISCATED is terminal (I1), CONFISCATE is universally reachable (I2), and the transition function is deterministic (I3).

3. SEIZED to FROZEN and FROZEN to RESTRICTED direct transitions are excluded by design, with legal and complexity-reduction justifications.

4. The synchronization protocol (lock, then update, then unlock) preserves cross-chain consistency for the same asset while isolating other assets.

5. Preemptive locking prevents concurrent regulatory actions on the same asset (NOT double-spend prevention that is handled at a different layer).

6. The `regulatory_homomorphism` theorem establishes that after `sync`, all connected chains agree on the new regulatory state.

7. The `valid_state_preservation` theorem establishes that `sync` preserves the global validity invariant: `consistent_state` AND no outstanding locks. This closes the inductive invariant.

8. The `multi_domain_preservation` locale is instantiated parametrically: for any finite domain set and valid global state, the abstract framework from `State_Preservation.thy` applies to the regulatory model.

What remains for future work: - Prove eventual consistency under the

partially synchronous network model (with message delays, redelivery, and out-of-order arrival) - (Optional) Connect to the `state_preservation` locale with identity state/action maps for the uniform-model case

end

theory *Priority-Resolution*
imports *Main*
begin

17 Priority-Based Deterministic Selection

A system where messages carry priorities from a linear order. Given a finite non-empty set of messages with injective priorities, there exists a unique highest-priority message, and selection is deterministic.

The injectivity assumption models tiebreaking: in practice, a composite key (e.g., authority level, timestamp, severity, node ID) ensures no two distinct messages share the same priority.

locale *priority-system* =
fixes *priority* :: 'm $\Rightarrow$ 'k::linorder
assumes *priority-injective*:
 $\llbracket \text{priority } m1 = \text{priority } m2 \rrbracket \implies m1 = m2$
begin

In a finite non-empty set with injective priorities, there exists a unique element with the maximum priority.

lemma *highest-priority-exists*:
assumes *finite S* **and** $S \neq \{\}$
shows $\exists! m. m \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m)$

proof –

have *fin-img*: *finite (priority ` S)* **using** *assms(1)* **by** *simp*
have *ne-img*: *priority ` S $\neq \{\}$* **using** *assms(2)* **by** *simp*
obtain *m* **where** *m-in*: $m \in S$ **and** *m-pri*: $\text{priority } m = \text{Max } (\text{priority ` } S)$
using *Max-in[OF fin-img ne-img]* **by** *auto*
have *m-max*: $\forall m' \in S. \text{priority } m' \leq \text{priority } m$

proof

fix *m'* **assume** $m' \in S$
hence $\text{priority } m' \in \text{priority ` } S$ **by** *simp*
hence $\text{priority } m' \leq \text{Max } (\text{priority ` } S)$ **using** *Max-ge[OF fin-img]* **by** *simp*
thus $\text{priority } m' \leq \text{priority } m$ **using** *m-pri* **by** *simp*

qed

show *?thesis*

proof (*rule ex1I[of - m]*)

show $m \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m)$
using *m-in m-max* **by** *auto*

next

fix *m2* **assume** $m2 \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m2)$

then have *m2-in*: $m2 \in S$ and *m2-max*: $\forall m' \in S. \text{priority } m' \leq \text{priority } m2$
 by *auto*
 from *m-max m2-in* have $\text{priority } m2 \leq \text{priority } m$ by *auto*
 from *m2-max m-in* have $\text{priority } m \leq \text{priority } m2$ by *auto*
 then have $\text{priority } m = \text{priority } m2$ using $\langle \text{priority } m2 \leq \text{priority } m \rangle$ by *auto*
 then show $m2 = m$ using *priority-injective* by *auto*
 qed
 qed

definition *select-highest* :: '*m* set $\Rightarrow$ '*m* option **where**
select-highest *S* =
 (if $S = \{\}$ then *None*
 else *Some* (*THE* *m*. $m \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m)$))

theorem *select-highest-deterministic*:
 assumes *fin*: *finite* *S* and *ne*: $S \neq \{\}$
 shows $\exists! m. \text{select-highest } S = \text{Some } m$
proof –
 from *ne* obtain *v* where *hv*: $\text{select-highest } S = \text{Some } v$
 unfolding *select-highest-def* by *simp*
 show ?thesis
proof (*rule* *ex1I*[*of* - *v*])
 show $\text{select-highest } S = \text{Some } v$ by (*rule* *hv*)
 next
 fix *w* assume $\text{select-highest } S = \text{Some } w$
 with *hv* show $w = v$ by *simp*
 qed
 qed

theorem *select-highest-in-set*:
 assumes *fin*: *finite* *S* and *sel*: $\text{select-highest } S = \text{Some } m$
 shows $m \in S$
proof –
 from *sel* have *ne*: $S \neq \{\}$
 unfolding *select-highest-def* by (*simp* *split*: *if-splits*)
 from *highest-priority-exists*[*OF* *fin ne*] have *uniq*:
 $\exists! x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x)$.
 from *theI*'[*OF* *uniq*] have *the-in*:
 $(\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x)) \in S$ by *auto*
 from *sel ne* have $m = (\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x))$
 unfolding *select-highest-def* by *simp*
 with *the-in* show ?thesis by *simp*
 qed

theorem *select-highest-is-max*:
 assumes *fin*: *finite* *S* and *ne*: $S \neq \{\}$ and *sel*: $\text{select-highest } S = \text{Some } m$
 shows $\forall m' \in S. \text{priority } m' \leq \text{priority } m$
proof –
 from *highest-priority-exists*[*OF* *fin ne*] have *uniq*:

```

   $\exists !x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x) .$ 
from theI'[OF uniq] have the-max:
   $\forall m' \in S. \text{priority } m' \leq$ 
     $\text{priority } (\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x))$ 
by auto
from sel ne have  $m = (\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x))$ 
unfolding select-highest-def by simp
with the-max show ?thesis by simp
qed

end

```

18 Deadlock-Free Locking with Timeout

A resource locking system where each lock has a bounded lifetime. After the timeout expires, the lock is considered released regardless of whether the holder explicitly released it. This models systems where Byzantine participants may hold locks indefinitely, but a timeout mechanism prevents permanent blocking.

The key property: no resource can be locked beyond its timeout, so no circular wait can persist indefinitely.

```

locale deadlock-free-locking =
  fixes timeout :: nat
  assumes timeout-positive:  $\text{timeout} > 0$ 
begin

```

A lock is effective only if it has not expired. Once the current time exceeds the lock acquisition time plus timeout, the lock is no longer blocking.

```

definition lock-effective :: nat  $\Rightarrow$  nat  $\Rightarrow$  bool where
  lock-effective lock-time current-time  $\longleftrightarrow$   $\text{current-time} < \text{lock-time} + \text{timeout}$ 

```

Every lock eventually expires: for any lock acquired at time t , there exists a future time at which the lock is no longer effective.

```

theorem lock-eventually-expires:
   $\exists t'. t' \geq \text{current-time} \wedge \neg \text{lock-effective lock-time } t'$ 
proof
  show  $\text{lock-time} + \text{timeout} + \text{current-time} \geq \text{current-time} \wedge$ 
     $\neg \text{lock-effective lock-time } (\text{lock-time} + \text{timeout} + \text{current-time})$ 
  unfolding lock-effective-def by auto
qed

```

The timeout is bounded: the lock expires no later than $\text{lock-time} + \text{timeout}$ time units after acquisition.

```

lemma lock-expiry-bound:
  assumes  $\neg \text{lock-effective lock-time } t$ 
  shows  $t \geq \text{lock-time} + \text{timeout}$ 
  using assms unfolding lock-effective-def by auto

```

At the exact expiry moment, the lock is no longer effective.

lemma *lock-expired-at-timeout*:
 $\neg \text{lock-effective lock-time (lock-time + timeout)}$
unfolding *lock-effective-def* **by** *auto*

Before the timeout, the lock is effective.

lemma *lock-effective-before-timeout*:
assumes *current-time < lock-time + timeout*
shows *lock-effective lock-time current-time*
using *assms* **unfolding** *lock-effective-def* **by** *auto*

Deadlock freedom for single-resource locking: if each operation acquires at most one lock, and every lock has a bounded timeout, then no operation can be blocked indefinitely.

This follows directly from `lock_eventually_expires`: any lock that blocks an operation will expire within `timeout` time units. Since operations acquire single locks (no circular wait possible with single-resource locking), deadlock is structurally impossible.

theorem *deadlock-freedom*:
assumes *lock-effective lock-time current-time*
shows $\exists t'. t' \leq \text{lock-time} + \text{timeout} \wedge \neg \text{lock-effective lock-time } t'$
proof
show *lock-time + timeout ≤ lock-time + timeout* $\wedge$
 $\neg \text{lock-effective lock-time (lock-time + timeout)}$
using *lock-expired-at-timeout* **by** *auto*
qed
end

19 Fair Leader Starvation Freedom

A leader-based system where an honest leader always processes at least one pending request, and a fairness assumption guarantees that honest leaders appear periodically.

The fairness assumption is a sufficient condition that abstracts over the probabilistic guarantees of VRF-based leader election. Under $f < n / (3::'a)$ Byzantine faults, the probability that a Byzantine leader is elected k consecutive times is $(f / n)^k$, which decreases exponentially. The fairness bound k captures this as a deterministic assumption.

The key theorem: under the fairness assumption, any pending request is processed within a bounded number of epochs.

locale *fair-leader-system* =
fixes *leader-at :: nat ⇒ 'n*
and *is-honest :: 'n ⇒ bool*
and *pending :: nat ⇒ nat*
and *fairness-bound :: nat*

```

assumes fairness-bound-positive: fairness-bound > 0
and fair-leader:
   $\forall \text{epoch}. \exists e. \text{epoch} \leq e \wedge e < \text{epoch} + \text{fairness-bound} \wedge \text{is-honest} (\text{leader-at } e)$ 
and honest-progress:
   $\llbracket \text{is-honest} (\text{leader-at } e); \text{pending } e > 0 \rrbracket \implies \text{pending} (\text{Suc } e) < \text{pending } e$ 
and non-honest-bounded:
   $\text{pending} (\text{Suc } e) \leq \text{pending } e$ 
begin

```

Pending count is monotonically non-increasing.

```

lemma pending-monotone:
  assumes  $e1 \leq e2$ 
  shows  $\text{pending } e2 \leq \text{pending } e1$ 
  using assms
proof (induction rule: inc-induct)
  case base
  then show ?case by simp
next
  case (step e)
  have  $\text{pending } e2 \leq \text{pending} (\text{Suc } e)$  by (rule step.IH)
  also have  $\text{pending} (\text{Suc } e) \leq \text{pending } e$  using non-honest-bounded by auto
  finally show ?case .
qed

```

If there are pending requests, within *fairness-bound* epochs at least one will be processed (the pending count strictly decreases).

```

theorem starvation-bound:
  assumes pos:  $\text{pending epoch} > 0$ 
  shows  $\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + \text{fairness-bound} \wedge \text{pending} (\text{Suc } e) < \text{pending } e$ 
proof –
  from fair-leader obtain h where
    h-range:  $\text{epoch} \leq h \wedge h < \text{epoch} + \text{fairness-bound}$  and
    h-honest:  $\text{is-honest} (\text{leader-at } h)$ 
  by auto
  show ?thesis
  proof (cases pending h > 0)
    case True
    with h-honest have  $\text{pending} (\text{Suc } h) < \text{pending } h$  using honest-progress by auto
    with h-range show ?thesis by auto
  next
    case False
    then have  $\text{pending } h = 0$  by simp
    — pending dropped from >0 (at epoch) to 0 (at h): find a strict decrease step
    have key:  $\forall n. 0 < n \longrightarrow \text{pending} (\text{epoch} + n) = 0 \longrightarrow$ 
       $(\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + n \wedge \text{pending} (\text{Suc } e) < \text{pending } e)$ 
    proof (rule allI)

```

```

fix n
show  $0 < n \longrightarrow \text{pending}(\text{epoch} + n) = 0 \longrightarrow$ 
   $(\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + n \wedge \text{pending}(\text{Suc } e) < \text{pending } e)$ 
proof (induction n)
  case 0 show ?case by simp
next
  case (Suc k)
  show ?case
  proof (intro impI)
    assume hpk:  $\text{pending}(\text{epoch} + \text{Suc } k) = 0$ 
    show  $\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + \text{Suc } k \wedge \text{pending}(\text{Suc } e) < \text{pending } e$ 
    proof (cases  $k = 0$ )
      case True
      from hpk True have  $\text{pending}(\text{Suc } \text{epoch}) < \text{pending } \text{epoch}$  using pos by
simp
      thus ?thesis by (intro exI[of - epoch]) auto
    next
      case False
      then have kpos:  $0 < k$  by simp
      show ?thesis
      proof (cases  $\text{pending}(\text{epoch} + k) = 0$ )
        case True
        then have pk0:  $\text{pending}(\text{epoch} + k) = 0$  .
        from Suc.IH[THEN mp, OF kpos, THEN mp, OF pk0]
        obtain e' where  $\text{epoch} \leq e' \wedge e' < \text{epoch} + k \wedge \text{pending}(\text{Suc } e') < \text{pending}$ 
e'
          by auto
          thus ?thesis by auto
        next
          case False
          from False hpk have  $\text{pending}(\text{epoch} + \text{Suc } k) < \text{pending}(\text{epoch} + k)$ 
by simp
          thus ?thesis by (intro exI[of - epoch + k]) auto
        qed
      qed
    qed
  qed
  have dpos:  $0 < h - \text{epoch}$ 
  proof -
    have  $h \neq \text{epoch}$  using ph0 pos by auto
    with h-range(1) show ?thesis by arith
  qed
  have pd:  $\text{pending}(\text{epoch} + (h - \text{epoch})) = 0$ 
  proof -
    have  $\text{epoch} + (h - \text{epoch}) = h$  using h-range(1) by arith
    with ph0 show ?thesis by simp
  qed
  from key[rule-format, OF dpos, OF pd]

```

```

    obtain e' where he': epoch ≤ e' e' < epoch + (h - epoch) pending (Suc e') <
    pending e'
    by auto
    have epoch + (h - epoch) ≤ h using h-range(1) by arith
    with he' h-range(2) show ?thesis by auto
  qed
qed

```

Corollary: all pending requests are eventually processed. By induction on the pending count (which is a natural number and thus a well-founded decreasing measure), repeated application of `starvation_bound` drives the count to zero.

```

theorem eventual-completion:
  shows ∃ e-final. pending e-final = 0
proof -
  define P where
    P n ≡ ∀ epoch. pending epoch = n ⟶ (∃ e-final. pending e-final = 0)
  for n :: nat
  have all-P: ⋀n. P n
  proof -
    fix n show P n
    unfolding P-def
    proof (induction n rule: less-induct)
      case (less n)
      show ∀ epoch. pending epoch = n ⟶ (∃ e-final. pending e-final = 0)
      proof (intro allI impI)
        fix epoch assume eq: pending epoch = n
        show ∃ e-final. pending e-final = 0
        proof (cases n = 0)
          case True thus ?thesis using eq by auto
        next
          case False
          have pos: pending epoch > 0 using eq False by auto
          from starvation-bound[OF pos] obtain e where
            e-ge: epoch ≤ e and e-dec: pending (Suc e) < pending e
            by auto
          have lt: pending (Suc e) < n
            using pending-monotone[OF e-ge] eq e-dec by simp
          from less.IH[OF lt] show ?thesis
            unfolding P-def by blast
        qed
      qed
    qed
  qed
  from all-P[of pending 0] show ?thesis
    unfolding P-def by blast
qed
end

```

end

```
theory DQuencer-Instance
  imports Priority-Resolution Regulatory-Instance
begin
```

20 Authority Level and Action Severity

Regulatory authority hierarchy. International authorities (e.g., FATF) take precedence over national (e.g., SEC), which take precedence over regional authorities. This reflects the RCP framework’s jurisdictional priority model.

```
datatype authority-level = Regional | National | International
```

```
fun authority-rank :: authority-level  $\Rightarrow$  nat where
  authority-rank Regional = 1
| authority-rank National = 2
| authority-rank International = 3
```

```
lemma authority-rank-injective:
  authority-rank a1 = authority-rank a2  $\implies$  a1 = a2
by (cases a1; cases a2; auto)
```

Action severity determines priority among actions from the same authority level. Stronger enforcement actions (CONFISCATE, SEIZE) take precedence over weaker ones (RESTRICT, UNFREEZE). This reflects the legal principle that more conservative (asset-protective) actions should prevail when concurrent conflicting orders exist.

```
fun action-severity :: reg-action  $\Rightarrow$  nat where
  action-severity UNRESTRICT = 1
| action-severity UNFREEZE = 2
| action-severity RELEASE = 3
| action-severity RESTRICT = 4
| action-severity FREEZE = 5
| action-severity SEIZE = 6
| action-severity CONFISCATE = 7
```

```
lemma action-severity-injective:
  action-severity a1 = action-severity a2  $\implies$  a1 = a2
by (cases a1; cases a2; auto)
```

21 Priority Key

The priority key is a 4-tuple of natural numbers, using Isabelle’s built-in lexicographic order on products. No manual linorder instance registration is needed.

Components (highest to lowest significance):

1. **authority_rank**: higher authority = higher priority
2. **inverted_timestamp**: earlier timestamp = higher priority (inverted)
3. **action_severity**: stronger action = higher priority
4. **node_id**: deterministic tiebreaker (lower node ID = higher priority, so we invert to make larger = higher in the nat order)

For timestamp and **node_id** inversion, we use *max-val* – *actual-val* to convert "smaller is better" to "larger is better" for compatibility with the nat product order where larger = higher.

type-synonym *priority-key* = *nat* × *nat* × *nat* × *nat*

22 Extended Message Type

Extends **oss_message** from **Regulatory_Instance** with authority level and source node ID. Uses Isabelle record inheritance so that existing functions on **oss_message** remain applicable.

record *dq-message* = *oss-message* +
 dqm-authority-level :: *authority-level*
 dqm-source-node :: *nat*

23 Priority Construction

Construct a priority key from message fields. Uses two upper bounds for inversion: **max_time** for timestamps and **max_node** for node IDs.

definition *make-priority-key* ::
 nat ⇒ *nat* ⇒ *dq-message* ⇒ *priority-key*
where
 make-priority-key max-time max-node msg =
 (*authority-rank* (*dqm-authority-level* *msg*),
 max-time – *msg-timestamp* *msg*,
 action-severity (*msg-action* *msg*),
 max-node – *dqm-source-node* *msg*)

24 Node and System Model

datatype *node-behavior* = *Honest* | *Byzantine*

record *node-info* =
 ni-id :: *nat*
 ni-behavior :: *node-behavior*

definition *honest-nodes* :: node-info set $\Rightarrow$ node-info set **where**
honest-nodes ns = {n $\in$ ns. ni-behavior n = Honest}

definition *byzantine-nodes* :: node-info set $\Rightarrow$ node-info set **where**
byzantine-nodes ns = {n $\in$ ns. ni-behavior n = Byzantine}

lemma *honest-byzantine-partition*:

honest-nodes ns $\cup$ *byzantine-nodes* ns = ns

proof (rule set-eqI)

fix x

show x $\in$ *honest-nodes* ns $\cup$ *byzantine-nodes* ns $\longleftrightarrow$ x $\in$ ns

unfolding *honest-nodes-def* *byzantine-nodes-def*

by (cases ni-behavior x; auto)

qed

lemma *honest-byzantine-disjoint*:

honest-nodes ns $\cap$ *byzantine-nodes* ns = {}

unfolding *honest-nodes-def* *byzantine-nodes-def* **by** auto

25 D-quencer System Locale

The D-quencer system locale encapsulates the BFT assumption (($3::'a$) * f + 1 $\leq$ n) and system parameters.

locale *dquencer-system* =

fixes *nodes* :: node-info set

and *f-max* :: nat

and *lock-timeout* :: nat

and *fairness-bound* :: nat

and *max-time* :: nat

and *max-node* :: nat

assumes *finite-nodes*: finite *nodes*

and *bft-threshold*: card *nodes* $\geq$ 3 * *f-max* + 1

and *byzantine-bound*: card (*byzantine-nodes* *nodes*) $\leq$ *f-max*

and *nonempty-nodes*: *nodes* $\neq$ {}

and *timeout-positive*: *lock-timeout* > 0

and *fairness-positive*: *fairness-bound* > 0

begin

lemma *honest-majority*:

card (*honest-nodes* *nodes*) > 2 * *f-max*

proof –

have card *nodes* = card (*honest-nodes* *nodes*) + card (*byzantine-nodes* *nodes*)

using *honest-byzantine-partition* *honest-byzantine-disjoint* *finite-nodes*

by (metis card-Un-disjoint finite-Un)

with *bft-threshold* *byzantine-bound* **show** ?thesis **by** linarith

qed

```

lemma honest-nonempty:
  honest-nodes nodes  $\neq \{\}$ 
proof –
  have fin: finite (honest-nodes nodes)
    unfolding honest-nodes-def using finite-nodes by simp
  have pos:  $0 < \text{card } (\text{honest-nodes nodes})$ 
    using honest-majority by linarith
  with fin pos show ?thesis by (auto simp: card-eq-0-iff)
qed

end

```

26 Priority System Instantiation

We instantiate the `priority_system` locale from `Priority_Resolution.thy` with D-quencer message priorities.

The priority function maps messages to `priority_key` (which is `nat \<times> nat \<times> nat \<times> nat` with the built-in lexicographic order).

Injectivity follows from: distinct messages have either different authority levels, different timestamps, different actions, or different source nodes. The source node ID serves as the final tiebreaker ensuring no two distinct messages share the same key.

For a concrete message set with the injectivity precondition, the `priority_system` locale provides deterministic selection.

```

lemma priority-key-injectivity:
  assumes make-priority-key mt mn m1 = make-priority-key mt mn m2
    and msg-timestamp m1  $\leq$  mt and msg-timestamp m2  $\leq$  mt
    and dqm-source-node m1  $\leq$  mn and dqm-source-node m2  $\leq$  mn
  shows dqm-authority-level m1 = dqm-authority-level m2
     $\wedge$  msg-timestamp m1 = msg-timestamp m2
     $\wedge$  msg-action m1 = msg-action m2
     $\wedge$  dqm-source-node m1 = dqm-source-node m2
proof –
  from assms(1) have
    eq1: authority-rank (dqm-authority-level m1) = authority-rank (dqm-authority-level m2) and
    eq2: mt - msg-timestamp m1 = mt - msg-timestamp m2 and
    eq3: action-severity (msg-action m1) = action-severity (msg-action m2) and
    eq4: mn - dqm-source-node m1 = mn - dqm-source-node m2
    unfolding make-priority-key-def by auto
  from eq1 have al: dqm-authority-level m1 = dqm-authority-level m2
    using authority-rank-injective by auto
  from eq2 assms(2,3) have ts: msg-timestamp m1 = msg-timestamp m2
    by linarith
  from eq3 have act: msg-action m1 = msg-action m2
    using action-severity-injective by auto

```

```

from eq4 assms(4,5) have nd: dqm-source-node m1 = dqm-source-node m2
  by linarith
from al ts act nd show ?thesis by auto
qed

```

27 BFT Consensus Abstraction

We abstract the BFT consensus mechanism. Rather than formalizing BLS signatures and vote counting, we model the consensus as: given a set of messages and honest majority, the consensus output is the highest-priority valid message.

This is justified because:

1. Honest nodes follow the priority protocol
2. With $> 2/3$ honest nodes, the BFT consensus output reflects the honest majority's agreement
3. The honest majority will agree on the highest-priority message (since priority is a total order and deterministic)

definition *valid-dq-message* :: *global-state* $\Rightarrow$ *dq-message* $\Rightarrow$ *bool* **where**

```

valid-dq-message gs msg =
  (asset-exists gs (msg-source msg) (msg-asset-id msg)  $\wedge$ 
   ( $\exists$  s. get-reg-state gs (msg-source msg) (msg-asset-id msg) = Some s  $\wedge$ 
    reg-transition s (msg-action msg)  $\neq$  None))

```

A scalar encoding of the priority key for use with `sort_key`. Maps each message to a single natural number that preserves the lexicographic order on priority key components, given the bounds *max-t* and *max-n*.

definition *priority-scalar* ::

```

nat  $\Rightarrow$  nat  $\Rightarrow$  dq-message  $\Rightarrow$  nat

```

where

```

priority-scalar max-t max-n m =
  authority-rank (dqm-authority-level m) * (Suc max-t) * 8 * (Suc max-n) +
  (max-t - msg-timestamp m) * 8 * (Suc max-n) +
  action-severity (msg-action m) * (Suc max-n) +
  (max-n - dqm-source-node m)

```

definition *bft-select* ::

```

dq-message list  $\Rightarrow$  global-state  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  dq-message option

```

where

```

bft-select msgs gs max-t max-n =
  (let valid-msgs = filter (valid-dq-message gs) msgs;
   ranked = sort-key (priority-scalar max-t max-n) valid-msgs
   in if ranked = [] then None else Some (last ranked))

```

28 Deadlock-Free Locking Instantiation

We instantiate the `deadlock_free_locking` locale with the D-quencer's lock timeout. This provides:

1. Every lock expires within `lock_timeout` time units
2. Byzantine nodes that hold locks indefinitely are handled by forced timeout release
3. Deadlock freedom: no asset can be permanently blocked

context *dquencer-system*
begin

The D-quencer system's lock timeout satisfies the `deadlock_free_locking` locale.

interpretation *dq-locking: deadlock-free-locking lock-timeout*
by *unfold-locales (rule timeout-positive)*

Concrete deadlock freedom for the D-quencer: any locked asset will be unlocked within `lock_timeout` time.

corollary *dq-deadlock-freedom:*
assumes *dq-locking.lock-effective lock-time current-time*
shows $\exists t'. t' \leq \text{lock-time} + \text{lock-timeout} \wedge$
 $\neg \text{dq-locking.lock-effective lock-time } t'$
using *dq-locking.deadlock-freedom[OF assms]* .

Byzantine lock resistance: even if a Byzantine node acquires a lock and never releases it, the lock expires by timeout.

corollary *byzantine-lock-expires:*
 $\exists t'. \neg \text{dq-locking.lock-effective lock-time } t'$
using *dq-locking.lock-eventually-expires by auto*

end

29 Fair Leader Starvation Freedom Instantiation

We instantiate the `fair_leader_system` locale with the D-quencer's leader election and pending request processing.

The fairness assumption abstracts VRF randomness: within any `fairness_bound` consecutive epochs, at least one epoch has an honest leader. Under $f < n / (3::'a)$, the probability of *fairness-bound* consecutive Byzantine leaders is $(f / n)^{\text{fairness-bound}} < (1 / (3::'a))^{\text{fairness-bound}}$.

For concrete instantiation, we need a leader schedule and pending count function that satisfy the locale assumptions. These are provided as parameters in the instantiation context.

```

locale dquencer-liveness = dquencer-system +
  fixes leader-schedule :: nat  $\Rightarrow$  node-info
    and pending-count :: nat  $\Rightarrow$  nat
  assumes fair-leader:
     $\forall$  epoch.  $\exists e$ . epoch  $\leq e \wedge e < \text{epoch} + \text{fairness-bound} \wedge$ 
      ni-behavior (leader-schedule e) = Honest
    and honest-processes:
       $\llbracket \text{ni-behavior } (\text{leader-schedule } e) = \text{Honest}; \text{pending-count } e > 0 \rrbracket$ 
       $\implies \text{pending-count } (\text{Suc } e) < \text{pending-count } e$ 
    and pending-non-increasing:
      pending-count (Suc e)  $\leq$  pending-count e
begin

interpretation dq-fair: fair-leader-system
  leader-schedule  $\lambda n$ . ni-behavior n = Honest pending-count fairness-bound
proof unfold-locales
  show  $0 < \text{fairness-bound}$  using fairness-positive .
next
  show  $\forall$  epoch.  $\exists e$ . epoch  $\leq e \wedge e < \text{epoch} + \text{fairness-bound} \wedge$ 
    ni-behavior (leader-schedule e) = Honest
    using fair-leader .
next
  fix e
  assume ni-behavior (leader-schedule e) = Honest  $0 < \text{pending-count } e$ 
  then show pending-count (Suc e)  $<$  pending-count e
    using honest-processes by auto
next
  fix e
  show pending-count (Suc e)  $\leq$  pending-count e
    using pending-non-increasing .
qed

```

Concrete starvation freedom for the D-quencer: if there are pending regulatory requests, at least one will be processed within `fairness_bound` epochs.

```

corollary dq-starvation-bound:
  assumes pending-count epoch  $> 0$ 
  shows  $\exists e$ . epoch  $\leq e \wedge e < \text{epoch} + \text{fairness-bound} \wedge$ 
    pending-count (Suc e)  $<$  pending-count e
  using dq-fair.starvation-bound[OF assms] .

```

end

30 Combined Safety and Liveness

This section connects Property 1 (cross-domain state preservation, safety) with Property 2 (determinism + liveness).

Property 1 guarantees: if sync is executed correctly, the resulting state

preserves cross-domain consistency.

Property 2 guarantees: under Byzantine faults, D-quencer deterministically selects actions, locks are bounded by timeout, and all pending requests are eventually processed.

Combined: in a decentralized Byzantine environment, cross-domain regulatory state is synchronized deterministically, without deadlock, and without starvation.

The combined theorem ties `valid_state_preservation` from `Regulatory_Instance.thy` with the liveness guarantees from this theory.

theorem *combined-safety-liveness*:

```

assumes valid: valid-state gs
and exists: asset-exists gs source aid
and current: get-reg-state gs source aid = Some s
and trans: reg-transition s action = Some s'
and not-locked:  $\neg$  is-locked gs aid
and fin: finite (connected-chains gs aid)
shows  $\exists gs'. \text{sync source action aid gs} = \text{Some gs}' \wedge \text{valid-state gs}'$ 
proof –
from not-locked obtain gs-locked where
  lock: acquire-lock gs aid = Some gs-locked
using lock-acquire-success by auto
from valid current trans lock have
   $\exists gs'. \text{sync source action aid gs} = \text{Some gs}'$ 
unfolding sync-def
using exists current trans lock
by (auto simp: get-reg-state-def get-asset-state-def
      split: option.splits
      intro!: exI)
then obtain gs' where synced: sync source action aid gs = Some gs'
by auto
have valid-state gs'
using valid-state-preservation[OF valid exists current trans synced fin] .
with synced show ?thesis by auto
qed

```

Summary of what Properties 1 + 2 together establish:

1. **Safety** (Property 1): After synchronization, all connected chains agree on the regulatory state. The global validity invariant is preserved.
2. **Determinism** (Property 2): Conflicting regulatory actions are resolved by a total order on priority keys. The BFT consensus output is unique.
3. **Deadlock freedom** (Property 2): No asset can be permanently locked, even if Byzantine nodes refuse to release locks. Timeout-based forced release bounds the lock duration.

4. **Starvation freedom** (Property 2): Under the fair leader assumption, every pending regulatory request is processed within a bounded number of epochs.

What remains: - Property 3: compositional assurance across heterogeneous verification regimes (Canton + OSS + EVM) - Refinement: formal correspondence between these models and the Go/Solidity implementation

end

31 Acknowledgments

The regulatory state model (five states, seven actions, and the transition exclusions) is motivated by the Regulatory Compliance Protocol (RCP), a framework developed by the Oraclizer research team that systematically classifies requirements from 15 global financial regulatory authorities for tokenized capital markets. RCP is currently proposed as an Informational Ethereum Improvement Proposal (EIP) in Draft status.

A companion paper presenting these proofs together with the surrounding context is available as [arXiv:2604.03844](https://arxiv.org/abs/2604.03844) [1], which discusses the mechanized proofs, the safety-liveness combination structure, the relation to Lochbihler’s Merkle Functor pattern, and the application context in the regulatory state synchronization domain.

References

- [1] J. Kim (for the Oraclizer Core Team), “Safety and Liveness of Cross-Domain State Preservation under Byzantine Faults: A Mechanized Proof in Isabelle/HOL,” [arXiv:2604.03844](https://arxiv.org/abs/2604.03844) [cs.CR], 2026. <https://arxiv.org/abs/2604.03844>.